



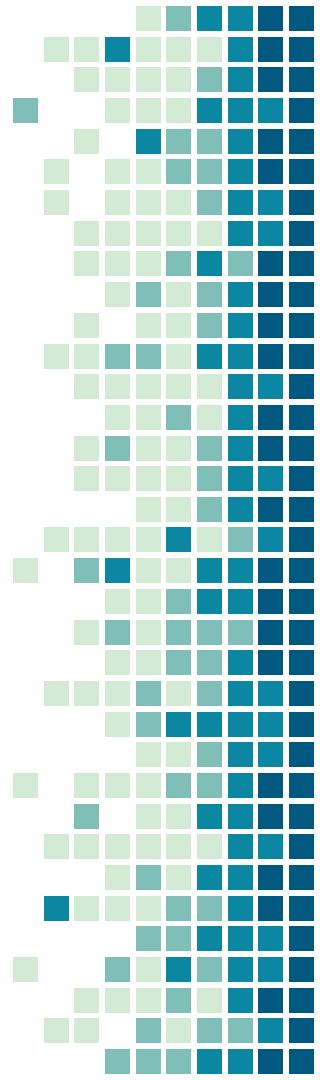
CdL in Informatica – A.A. 2024 – 2025

Programmazione 1 – Modulo 2

Lezione 13
3/12/2024

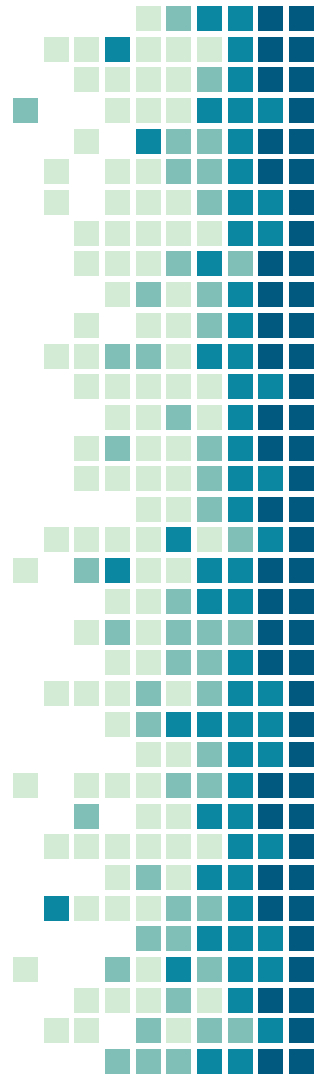
Andrea Loddo

Federico Meloni - Alessandra Perniciano - Fabio Pili



Argomenti

- Ricorsione
- Utility
 - Compilazione condizionale
 - Macro predefinite
 - Assert
 - Debugger



La ricorsione

La ricorsione

La ricorsione

La ricorsione

La ricorsione

La ricorsione

La ricorsione

La ricorsione

La ricorsione

3

3

3

3

3

Ricorsione

- **Algoritmo ricorsivo:** definito in termini di sé stesso
 - un algoritmo, per definizione, deve terminare: non può invocare sé stesso all'infinito
 - quindi: deve prevedere una condizione di terminazione, chiamata **caso base**
- Il caso base è definito in termini semplici (non ricorsivi)
- **Funzione ricorsiva:** richiama **sé stessa**
 - generalmente su un sottoinsieme dei dati del problema
- La funzione ricorsiva sa risolvere direttamente solo casi particolari di un problema detti **casi base**
 - se viene invocata passandole dei dati che costituiscono uno dei casi base, allora **restituisce un risultato**;
 - se invece viene chiamata passandole dei dati che **NON** costituiscono uno dei casi di base, allora chiama sé stessa (**passo ricorsivo**) con un insieme ridotto di dati.

Esempio: il fattoriale di un numero

- Il fattoriale di un numero naturale N è definito come il prodotto tra N e il fattoriale di N - 1.
- Il fattoriale di 0 è definito come 1 (caso base).

$$n! = \begin{cases} 1 & \text{se } n = 0 \\ n * (n - 1)! & \text{altrimenti} \end{cases}$$

```
int fattoriale (int n)
{
    if (n == 0)
        return 1;
    else
        return n * fattoriale(n-1);
}
```

Il fattoriale in esecuzione

```
int main() {  
    int f = fatt(3);  
}
```

```
int fatt (int n) {  
    if (n == 0)  
        return 1;  
    else  
        return n * fatt(n-1);  
}
```

n = 3

2

```
int fatt (int n) {  
    if (n == 0)  
        return 1;  
    else  
        return n * fatt(n-1);  
}
```

n = 2

1

```
int fatt (int n) {  
    if (n == 0)  
        return 1;  
    else  
        return n * fatt(n-1);  
}
```

n = 1

```
int fatt (int n) {  
    if (n == 0)  
        return 1;  
    else  
        return n * fatt(n-1);  
}
```

n = 0

3*2

2*1

1*1

1

0

$$3! = 3*2! = 3*2*1! = 3*2*1 = 6$$

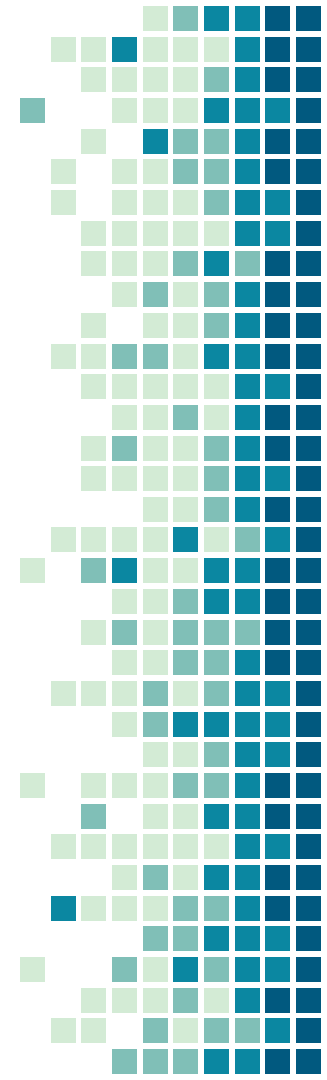
Il fattoriale

In codice (versione iterativa equivalente):

```
int fattoriale (int n)
{
    int i, ris;
    ris = 1;

    for( i = 1; i <=n; i++ )
        ris = ris * i;

    return ris;
}
```



Tipi di ricorsione

- ricorsione **diretta**: una funzione che invoca sé stessa;
- ricorsione **indiretta**: una funzione f invoca un'altra funzione g , la quale invoca ricorsivamente la funzione f ;
- ricorsione **lineare**: una singola invocazione ricorsiva all'interno della funzione;
- ricorsione **non lineare**: più invocazioni ricorsive all'interno della funzione;
- ricorsione **di coda**: la chiamata ricorsiva è l'ultima istruzione della funzione.
- NB: per ogni funzione ricorsiva, esiste una funzione iterativa equivalente.

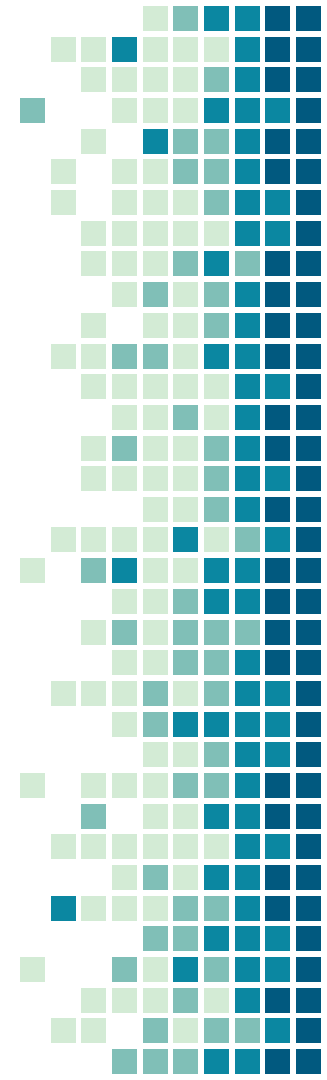
Pro/contro

Vantaggi:

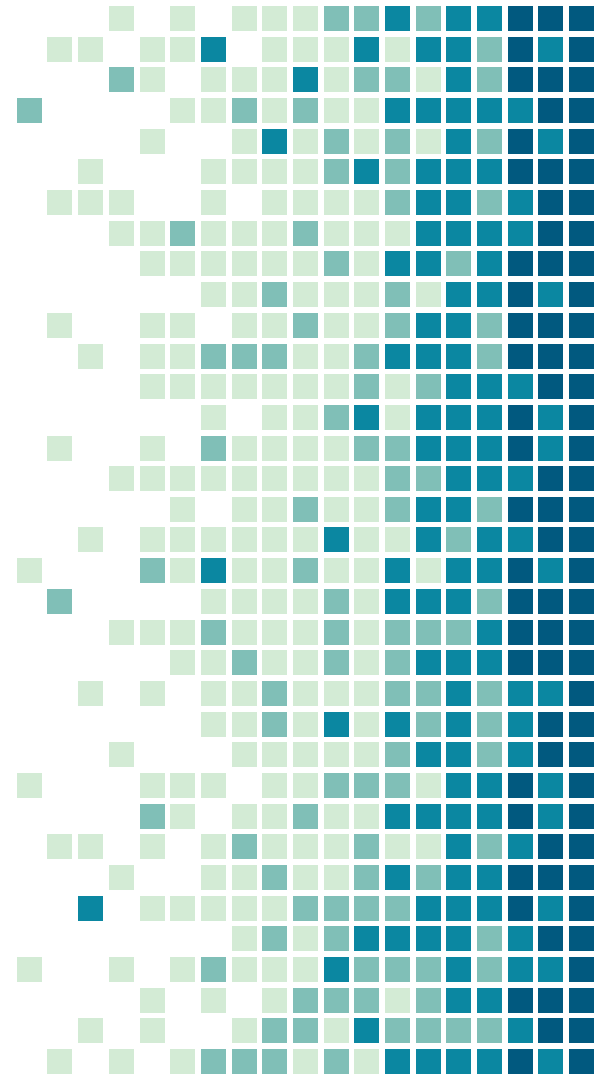
- Consente di definire in termini semplici e in maniera elegante numerose funzioni, anche molto complesse;
- Applicazioni non solo teorico-matematiche: la maggior parte degli algoritmi di ordinamento e delle strutture dati di ricerca è definita in termini ricorsivi.

Svantaggi:

- Poco efficiente (in termini di tempo di calcolo e spazio occupato)
- Ad ogni chiamata viene salvato il **contesto** della funzione, con l'attivazione di un nuovo stack frame.
 - Quindi: istanziando nuovamente le variabili locali non statiche e i parametri

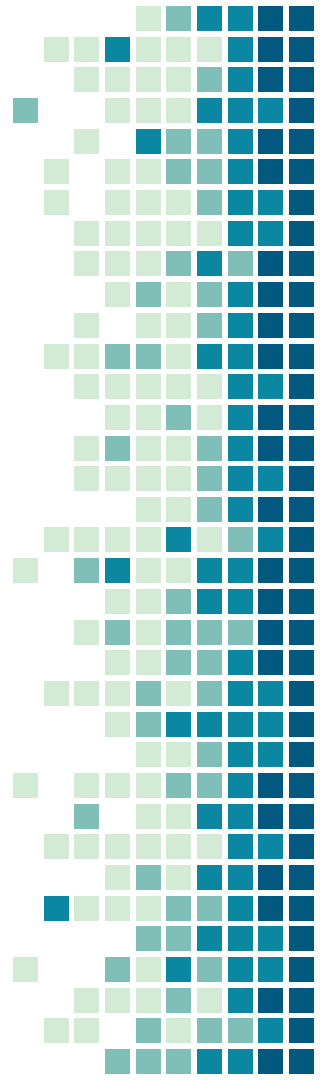


Compilazione condizionale



Compilazione condizionale

- Il preprocessore del linguaggio C fornisce ulteriori strumenti di utilità.
- Per esempio, è possibile compilare selettivamente il codice, includendo o escludendo parti di codice a seconda delle necessità.
- L'utilizzo più comune è quello di scrivere codice che possa essere compilato correttamente sotto differenti piattaforme, scegliendo di volta in volta le istruzioni o le chiamate più adatte sulla base della piattaforma target.
- Alternativamente si utilizza per il debug, solitamente per regolare la quantità di informazioni che vengono visualizzate sul terminale.



Direttive #if #elif #else #endif

- Esempio: si vuol verificare la versione dello standard in uso:

```
#if __STDC_VERSION__ >= 199901L
// standard C99
for(int i=0; i<10; i++)
    printf("%d", i);
#else
// standard non C99
int i;
for(i=0; i<10; i++)
    printf("%d", i);
#endif
```

- Esempio: si vuole eseguire del codice differente a seconda del valore di una MACRO.

- Se la costante `__STDC_VERSION__` è maggiore di 199901L, verranno compilate le istruzioni del ramo `#if`, altrimenti quelle del ramo `#else`.
- `__STDC_VERSION__` è una MACRO predefinita, vale 199409L per il C89 e 199901L per il C99.

```
#define ENG_US 1
#define ENG_UK 2
#define ITA 3

#define LANGUAGE ENG_UK

int main() {
    #if LANGUAGE==ENG_US
        printf("Language: american english\n");
    #elif LANGUAGE==ENG_UK
        printf("Language: british english\n");
    #else
        printf("Linguaggio: italiano\n");
    #endif

    return 0;
}
```

Direttive #ifdef e #ifndef

- Le direttive **#ifdef** e **#ifndef** non valutano il valore della costante, ma verificano solamente se la costante è definita o meno.

```
#ifdef NOME_COSTANTE
    codice da eseguire se NOME_COSTANTE è stata definita
#endif
```

```
#ifndef NOME_COSTANTE
    codice da eseguire se NOME_COSTANTE non è stata definita
#endif
```

- Utilizzo tipico: garantire la **portabilità** del codice.

```
/* Esempio: la funzione system ha un parametro differente per la pulizia
   dell'output a seconda della piattaforma */
#ifdef _WIN64 // eseguire codice per Windows
    printf("Sono su Windows");
    system("cls");
#elif __APPLE__ // eseguire codice per piattaforme Apple
    printf("Sono su Mac OS");
    system("clear");
#elif __linux__ // eseguire codice per Linux
    printf("Sono su Linux");
    system("clear");
#endif
```

Macro predefinite

In C, esistono due MACRO standard utilizzabili per indicare se un programma è terminato correttamente oppure no:

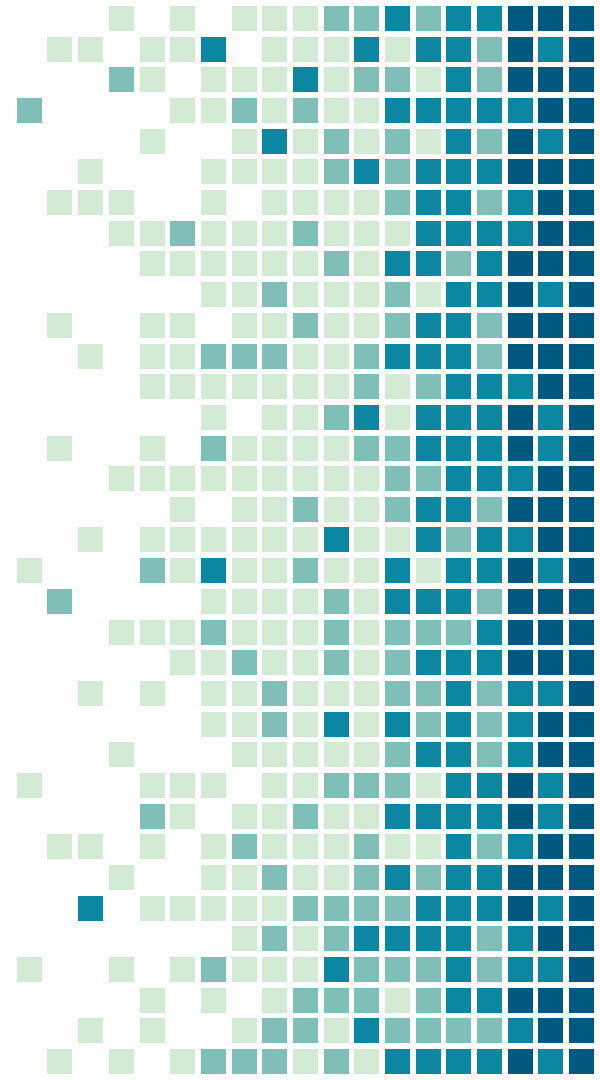
- **EXIT_SUCCESS** per indicare che il programma è terminato con successo;
- **EXIT_FAILURE** per indicare che il programma è terminato in modo anomalo.

Esempi:

```
int main() {  
    ...  
    return EXIT_SUCCESS;  
}
```

```
FP *filePointer = NULL;  
  
filePointer = fopen("file.txt", "w");  
  
if( filePointer == NULL)  
    exit(EXIT_FAILURE);
```

Assert



Assert

- Spesso può essere necessario verificare che certe condizioni siano verificate a runtime, soprattutto per testare il vostro programma durante la fase di scrittura / debug;
- La assert è una macro predisposta per il **debug**, definita in **assert.h** :

```
/* Sintassi della assert */  
void assert( int condizione );
```

- La condizione viene valutata a **runtime**:
 - se è diversa da 0, il programma continua.
 - Se è uguale a 0: il programma termina con un errore, mostrato sullo standard error, contenente la condizione stessa, il nome del file e il numero della riga dove è stata valutata l'assert.
- Tutte le **assert** vengono disabilitate in automatico quando si compila ed esegue in modalità **Release**.

Assert: esempio

```
#include <stdio.h>
#include <assert.h>
#include <math.h>

int main(void) {
    double x = -1.0;
    assert(x >= 0.0);
    printf("sqrt(x) = %f\n", sqrt(x));
    return 0;
}
```

Assertion failed!

Program: C:\Users\loand\Desktop\Test\cmake-build-debug\Test.exe

File: C:/Users/loand/Desktop/Test/main.c, Line 8

Expression: x >= 0.0

Process finished with exit code 3

Assert

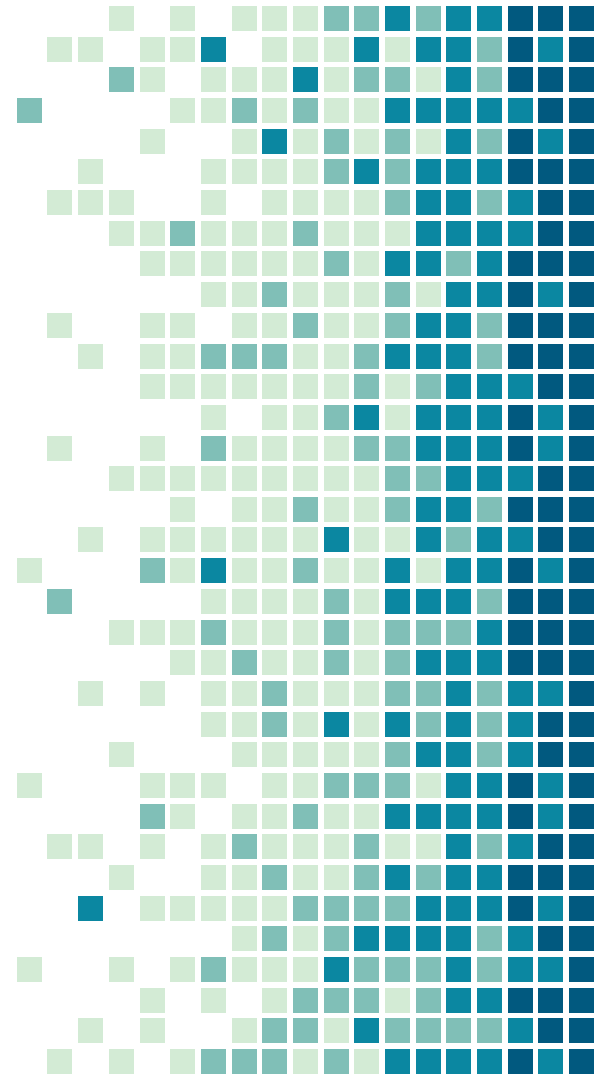
È buona norma accompagnare la condizione dell'assert con una stringa di errore, in modo da essere più facile riconoscere l'entità dell'errore:

```
#include <stdio.h>
#include <assert.h>
#include <math.h>

int main(void) {
    double x = -1.0;
    assert(x >= 0.0 && "X deve essere positivo");
    printf("sqrt(x) = %f\n", sqrt(x));
    return 0;
}
```

La stringa viene valutata come true, quindi non inficia il nostro controllo, ma può risultare utile nella stampa dell'errore.

Formattazione stampa



ASCII escape code

Il C ci permette di andare ad abbellire il testo con la formattazione generale che siamo abituati a vedere negli editor di testo:

- Grassetto
- Sottolineato
- Background
- Etc

Questi stili sono gestiti dagli escape code ASCII che noi andiamo a definire come macro:

```
#define BOLD "\e[1m"  
#define RESET "\e[0m"
```

ASCII escape code

Per utilizzarli possiamo metterli all'inizio e/o alla fine:

```
#define BOLD "\e[1m"  
#define RESET "\e[0m"
```

```
printf(BOLD "Questo testo è in grassetto" RESET);
```

Finché non andiamo a resettare il testo questo rimarrà con la stessa formattazione che abbiamo impostato precedentemente

Alcuni esempi:

```
#define RED "\e[0;31m"  
#define GRN "\e[0;32m"  
#define YEL "\e[0;33m"  
#define BLU "\e[0;34m"
```

```
#define BOLD "\e[1m"  
#define UNDERLINE "\e[4m"  
#define STRIKE "\e[9m"  
#define REVERSE "\e[7m"
```

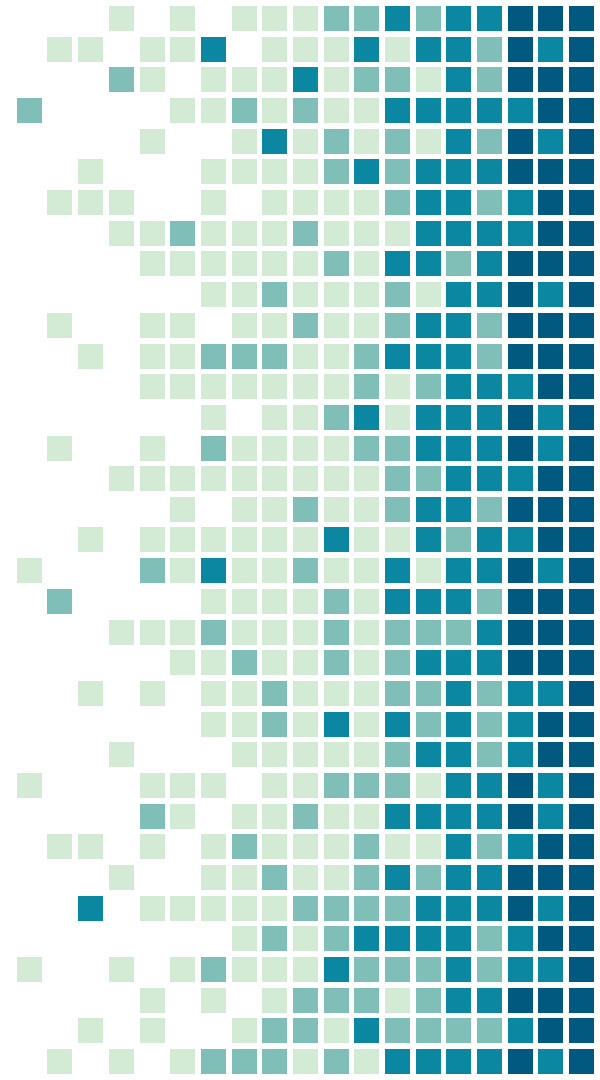
ASCII escape code

Potete trovare una lista esaustiva (per gli scopi di PR1) dei vari escape code al seguente link con un esempio di programma che permette di testare i vari colori e formattazioni:

<https://gist.github.com/federicheddu/036ddc1624c12c073d1d481f3044628a>

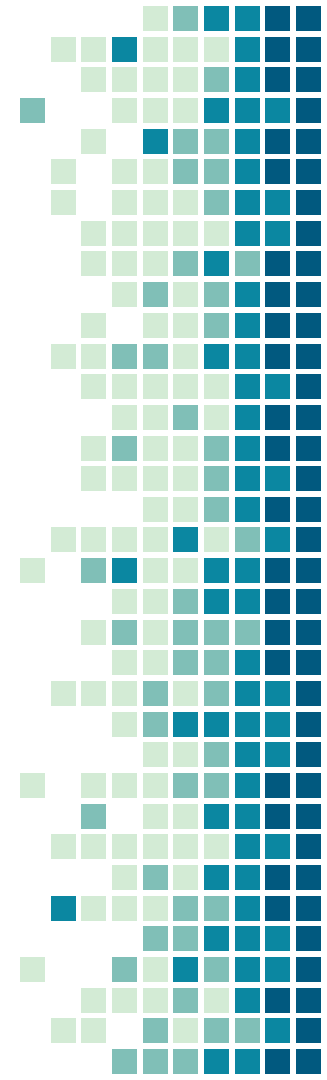


Debugger



Debugger

- In informatica il debugger è un programma/software specificatamente progettato per l'analisi e l'individuazione dei bug (debugging), ovvero errori di programmazione interni al codice di altri programmi.
- Errore comune: sottovalutarne l'utilità.
- GDB (GNU debugger): installato di default insieme a CLion + (GCC || MinGW)



Utilizzare il debugger

- Di default, CLion è impostato in modalità debug, quando creiamo un progetto

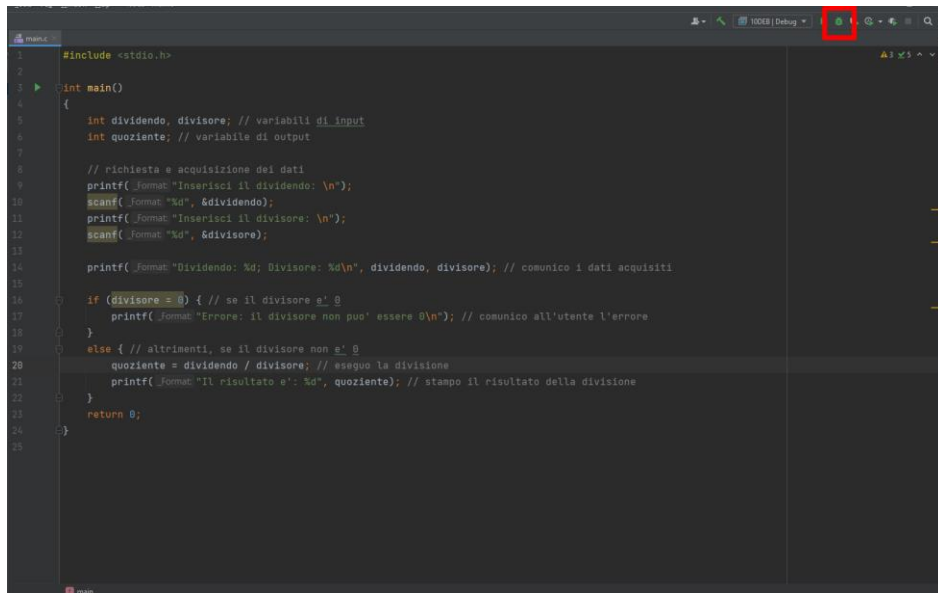


- In pratica, abbiamo una "configurazione" dedicata al *debugging*.
- Quando si compila utilizzando questa configurazione, si genera un eseguibile su cui è possibile utilizzare il **debugger**.
- L'altra modalità è detta "Release": la vedrete nel corso di Algoritmi e Strutture Dati.
- La differenza tra le due modalità risulta rilevante solo su progetti di grosse dimensioni.

Utilizzare il debugger

Il debugger permette di:

- Individuare la linea di codice dove il programma genera un errore (SIGFPE o SIGSEGV);
- Bloccare il flusso di esecuzione del programma a una determinata riga;
- Visualizzare, step by step, come le istruzioni eseguite modificano lo stato del programma (es: valori assunti dalle variabili).



```
#include <stdio.h>

int main()
{
    int dividendo, divisore; // variabili di input
    int quoziente; // variabile di output

    // richiesta e acquisizione dei dati
    printf("Inserisci il dividendo: \n");
    scanf("%d", &dividendo);
    printf("Inserisci il divisore: \n");
    scanf("%d", &divisore);

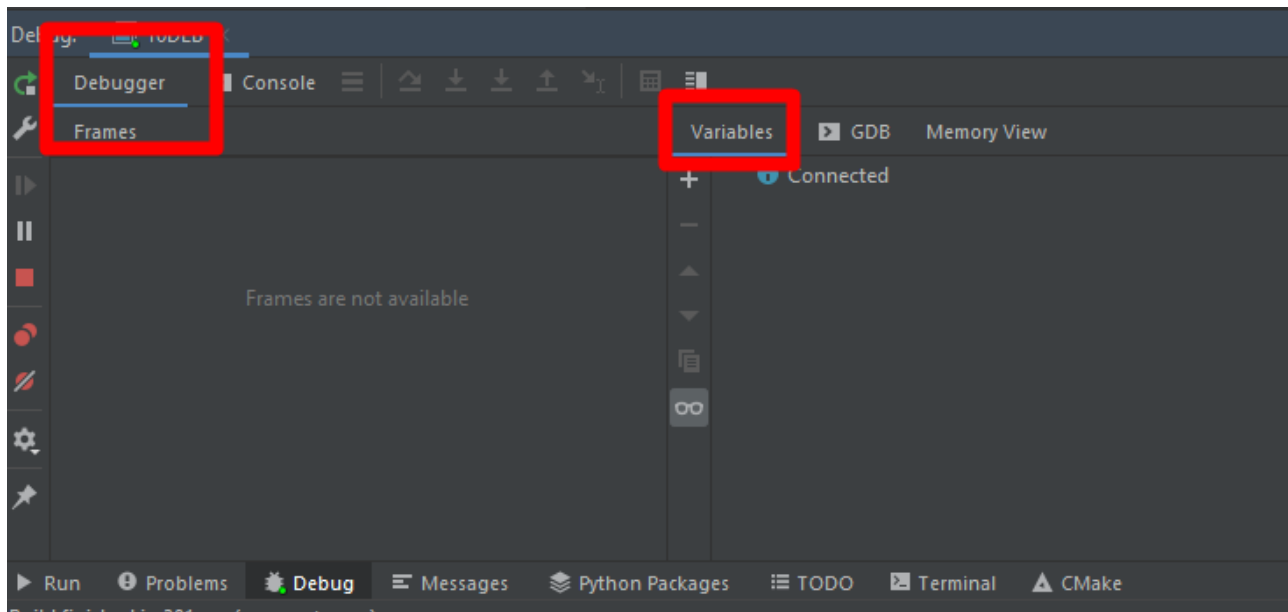
    printf("Dividendo: %d, Divisore: %d\n", dividendo, divisore); // comunico i dati acquisiti

    if (divisore == 0) { // se il divisore e' 0
        printf("Errore: il divisore non puo' essere 0\n"); // comunico all'utente l'errore
    }
    else { // altrimenti, se il divisore non e' 0
        quoziente = dividendo / divisore; // eseguo la divisione
        printf("Il risultato e': %d", quoziente); // stampo il risultato della divisione
    }

    return 0;
}
```

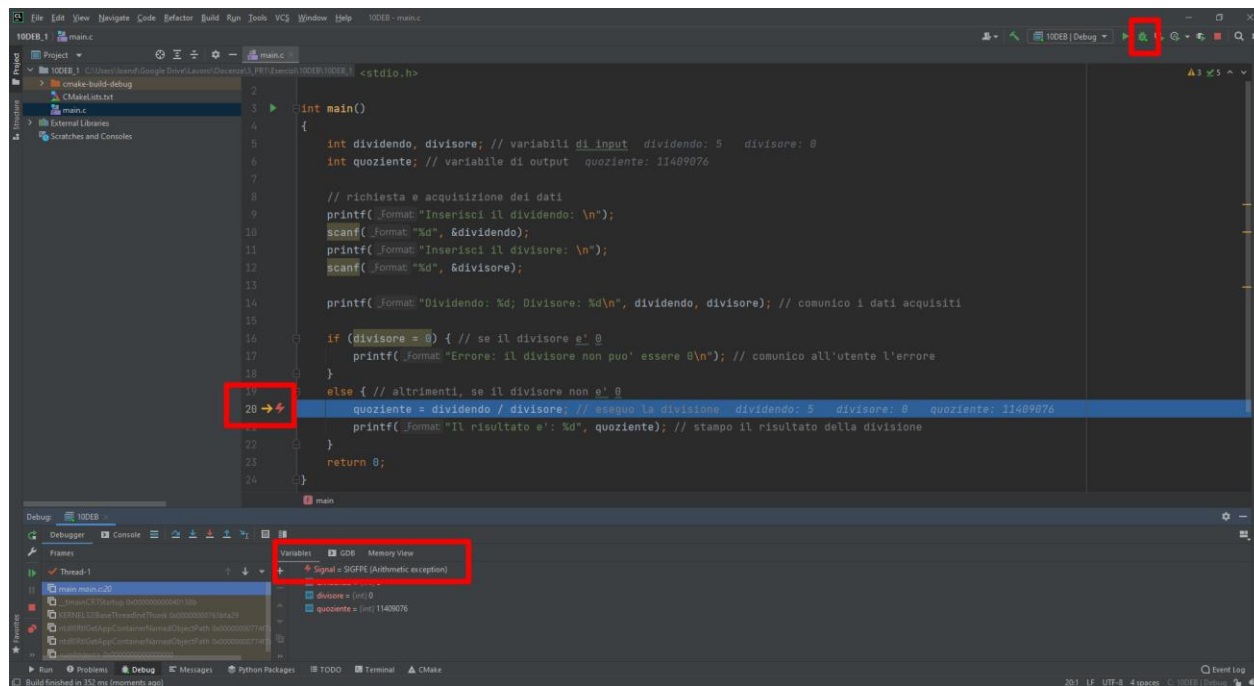
Utilizzare il debugger

- Durante il debugging, è possibile visualizzare delle schermate che mettono in evidenza caratteristiche e stato del programma in esecuzione.
- Nel nostro caso ci serviranno due schermate: “Debugger” e “Variables”, che compaiono in basso, quando lanciamo il debugger.



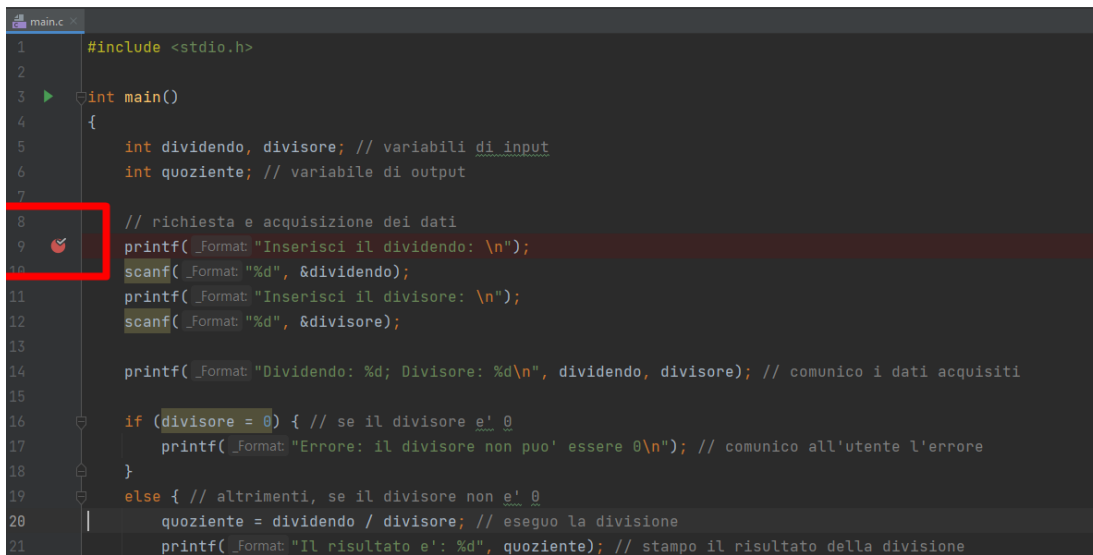
Utilizzare il debugger

- Per individuare in quale linea di codice accade un errore a runtime, è sufficiente lanciare il debugger (riquadro in alto a destra)
- Quando viene lanciato l'errore a runtime, viene visualizzato nel riquadro "Variables" in basso
- Viene evidenziata la riga che ha causato l'errore a runtime (nell'esempio la riga 20).



Utilizzare il debugger

- È anche possibile visualizzare lo stato delle variabili presenti nel programma durante l'esecuzione dello stesso. Per fare ciò, è necessario prima di tutto inserire un **breakpoint** nella riga desiderata.
- Il breakpoint indica la riga in cui il debugger dovrà fermare l'esecuzione del programma e visualizzarne lo stato nella schermata “Variables”.
- Per inserire un breakpoint, è sufficiente cliccare nello spazio a destra del numero della riga desiderata (nell'esempio, il breakpoint è stato inserito nella riga 10).

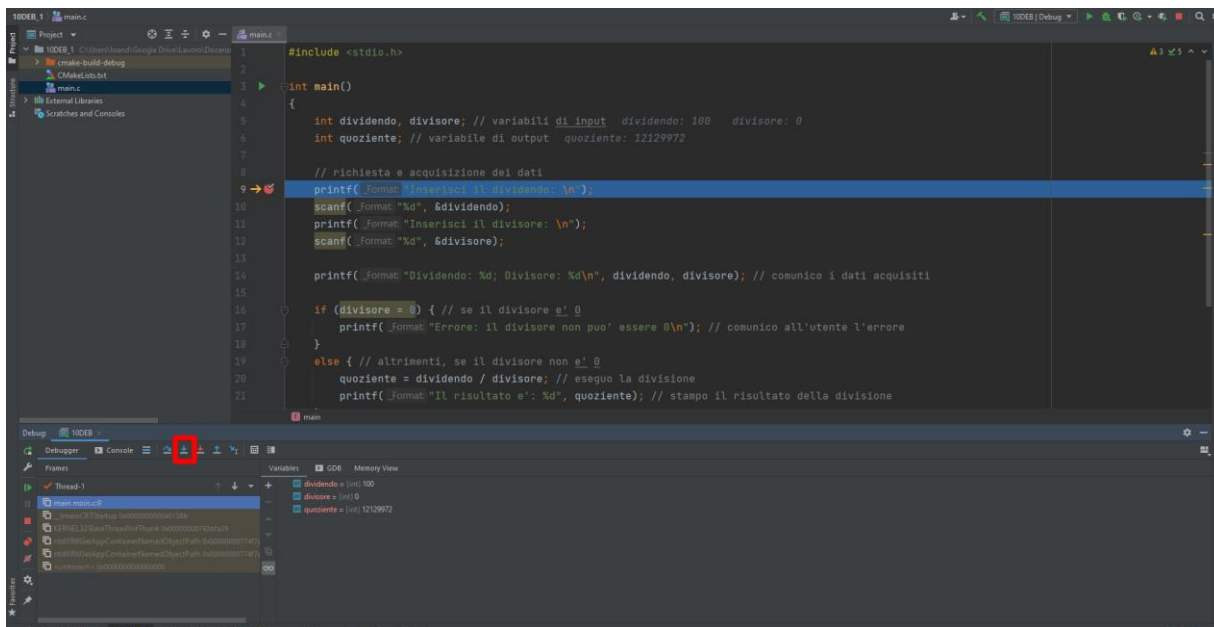


```
1 #include <stdio.h>
2
3 int main()
4 {
5     int dividendo, divisore; // variabili di input
6     int quoziente; // variabile di output
7
8     // richiesta e acquisizione dei dati
9     printf(_Format: "Inserisci il dividendo: \n");
10    scanf(_Format: "%d", &dividendo);
11    printf(_Format: "Inserisci il divisore: \n");
12    scanf(_Format: "%d", &divisore);
13
14    printf(_Format: "Dividendo: %d; Divisore: %d\n", dividendo, divisore); // comunico i dati acquisiti
15
16    if (divisore == 0) { // se il divisore e' 0
17        printf(_Format: "Errore: il divisore non puo' essere 0\n"); // comunico all'utente l'errore
18    }
19    else { // altrimenti, se il divisore non e' 0
20        quoziente = dividendo / divisore; // eseguo la divisione
21        printf(_Format: "Il risultato e': %d", quoziente); // stampo il risultato della divisione
```

The image shows a code editor window titled 'main.c'. The code is a C program for division. A red box highlights line 10, which contains the `scanf` statement for the divisor. A red bug icon, representing a breakpoint, is placed in the left margin next to line 10. The code includes comments in Italian and uses `_Format` for printf and scanf.

Utilizzare il debugger

- Una volta lanciato il debugger, è possibile, partendo dal breakpoint inserito, visualizzare come le operazioni eseguite modifichino le variabili step by step. Per fare ciò è sufficiente cliccare sul pulsante "Step into" (F7), che permette di eseguire solamente la riga successiva a quella dove è stata fermata l'esecuzione.
- Nella schermata "Variables" è possibile visualizzare le variabili dichiarate e il relativo valore.



Debugger su CLion e printf

- Il debugger integrato su CLion, quando mandato in esecuzione, non mostrerà le printf presenti nel vostro codice.
- Per evitare questo problema, potete inserire all'inizio del main() (e solo per fare debug), il seguente comando:

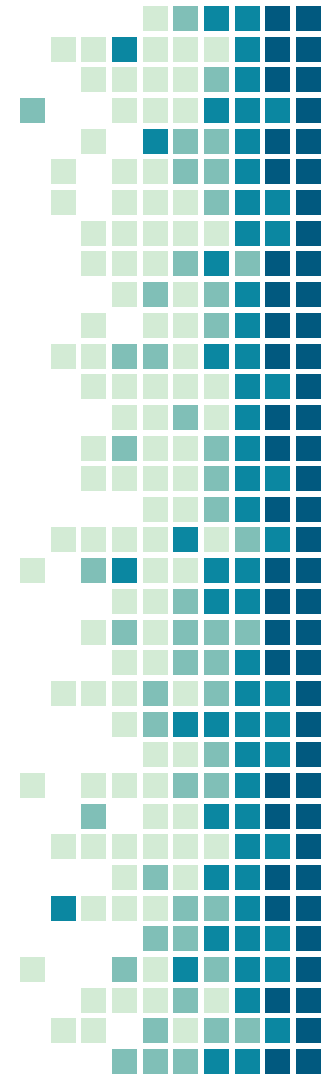
```
int main( ){  
    setvbuf(stdout, NULL, _IONBF, 0);  
  
    // Codice utente  
    ...  
}
```

FINE!

Domande?

Autovalutazione su Ricorsione

1. Quando una funzione si definisce ricorsiva?
2. Può una funzione ricorsiva non terminare mai?
3. Cosa rappresenta il passo base di una funzione ricorsiva?
4. Cosa genera l'assenza (o l'errata individuazione) del passo base?
5. Nella funzione ricorsiva fattoriale, come viene codificato il passo base?
6. Sono in grado di scrivere in completa autonomia la funzione ricorsiva per il calcolo del fattoriale?
7. Che tipo di ricorsione è quella presente nella funzione ricorsiva per il calcolo del fattoriale vista a lezione?
8. Quali sono i principali vantaggi della ricorsione?
9. E gli svantaggi?
10. Può una funzione ricorsiva essere diretta e di coda contemporaneamente?



Esercizi su Ricorsione

1. Scrivere una funzione ricorsiva che risolva l'esercizio 1;
2. Scrivere una funzione ricorsiva che sommi i numeri da 0 a n (passato in input);
3. Scrivere una funzione ricorsiva che calcoli l'n-esimo numero di Fibonacci, dove:
$$f(n) = f(n-1) + f(n-2)$$

(la serie di Fibonacci è quindi la seguente: (0) 1 1 2 3 5 8 13 21 ...)
4. Scrivere una funzione ricorsiva per stampare gli elementi di un array;
5. Scrivere una funzione ricorsiva che calcoli la somma degli elementi di un array;
6. Scrivere una funzione ricorsiva che, acquisito in input un array di n interi, restituisca in output **true** se il valore 10 è un elemento dell'array, **false** altrimenti.
7. Scrivere una funzione ricorsiva per determinare se una stringa è palindroma.
Suggerimento: una stringa è palindroma se i suoi caratteri estremi sono uguali e i restanti centrali costituiscono una stringa palindroma.
8. Scrivere una funzione ricorsiva per stampare una stringa al rovescio (non la deve invertire, ma solo stampare).
9. Scrivere una funzione ricorsiva che determini il minimo di un vettore di interi.

(suggerimento: per le funzioni sugli array, è possibile anche lavorare sugli indici)

